

Capital.com Public API - Operational Rules & Recovery

Part 5 for CapitalDemoBroker / Codex

Developer reference based on the official Capital.com Public API 1.0.0. This part consolidates rate limits, session/streaming behavior, hedging, activity statuses, trading-state safeguards, costs metadata, precision constraints and conservative retry/recovery rules.

1. Official API rate limits

Scope	Documented limit
General REST requests	Maximum 10 requests / second / user
Open positions or create orders	Maximum 1 request per 0.1 seconds / user
POST /session	Maximum 1 request / second / API key
DEMO POST /positions	1000 requests / hour
DEMO POST /workingorders	1000 requests / hour
WebSocket subscriptions	Maximum 40 instruments
API-key generation	Maximum 100 attempts / 24 hours

The official FAQ states that position/order requests exceeding the trading cadence can be rejected. The public page does not establish that a trading POST is safe to replay after a 429 or transport timeout. CapitalDemoBroker should therefore distinguish throttling before transmission from an ambiguous result after transmission.

2. REST and WebSocket session lifetime

- REST session: active for 10 minutes after last use.
- CST and X-SECURITY-TOKEN are valid for 10 minutes after last use.
- WebSocket session duration: 10 minutes; Capital.com instructs clients to use ping to keep it alive.
- Changing the financial account through PUT /session causes WebSocket streaming to fall off.

3. WebSocket connection and authentication

Connect:
wss://api-streaming-capital.backend-capital.com/connect

Authenticated subscription messages use:
destination
correlationId
cst = CST token
securityToken = X-SECURITY-TOKEN
payload

Operational rule:
keep the connection/session alive with ping at least once within
the documented 10-minute lifetime window.

After disconnect/reconnect, the public documentation provides subscription mechanics but does not document a global event sequence number guaranteeing gap detection across all streams. A safe recovery design should therefore refresh authoritative REST state (positions and working orders) before resuming strategy transitions.

4. Hedging mode - mandatory preflight for this strategy

GET /api/v1/accounts/preferences

Response includes:
{

```

    "hedgingMode": false,
    "leverages": { ... }
}

PUT /api/v1/accounts/preferences
{
  "hedgingMode": true
}

```

Capital.com explicitly identifies hedgingMode as the account trading-mode flag and allows it to be changed through PUT /accounts/preferences. A dual-sided strategy must read this value before starting a cycle and require the intended mode rather than assuming the account is hedged.

The trading contracts also state that guaranteed stops cannot be set when hedgingMode is enabled. This must be enforced before constructing position or working-order requests.

5. Activity/history statuses for reconciliation

GET /history/activity exposes broker activity and supports status filtering. The documented status enum is:

```

ACCEPTED
CREATED
EXECUTED
EXPIRED
REJECTED
MODIFIED
MODIFY_REJECT
CANCELLED
CANCEL_REJECT
UNKNOWN

```

Documented activity types include POSITION, WORKING_ORDER, EDIT_STOP_AND_LIMIT, SWAP and SYSTEM. Sources include CLOSE_OUT, DEALER, SL, SYSTEM, TP and USER. These values are useful when Recovery Manager must distinguish user actions, SL/TP actions and system/broker activity.

The maximum documented date range between from and to for GET /history/activity is one day.

6. Conservative retry classification

Situation	Safe broker-adapter behavior
Auth/session expired before a read	Re-authenticate, then retry the idempotent read
Read-only GET timeout	Retry with bounded backoff/rate limiting
Trading POST returned dealReference	Use GET /confirms/{dealReference}; do not submit another trade
Trading POST transport timeout, no dealReference	UNKNOWN outcome: reconcile broker state/history; do not blindly replay
PUT/DELETE returned dealReference	Confirm/reconcile before declaring local state final
429 before operation is known to be accepted	Respect limiter/backoff; trading operation still needs ambiguity handling
WebSocket disconnect	Reconnect, then reconcile REST positions/orders before strategy continuation
Activity status UNKNOWN	Enter Recovery/verification path, not automatic duplicate execution

7. Market state and trading hours

Market details expose openingHours and a marketStatus snapshot. New trades should be gated on the broker's current market state rather than a locally assumed weekday schedule. The API documentation includes states such as TRADEABLE and CLOSED in market responses.

The public contract does not establish that SL/TP editing or position closing is always possible while a market is closed. Treat such actions as broker-confirmed operations and surface rejection reasons rather than assuming execution.

8. Price/size precision and broker constraints

GET /markets/{epic} provides the metadata that should drive validation: tickSize, decimalPlacesFactor, minStepDistance, minDealSize, maxDealSize, minSizeIncrement, minGuaranteedStopDistance, minStopOrProfitDistance and maxStopOrProfitDistance.

Do not use an arbitrary Python round() policy as the broker contract. Quantize Decimal values to broker-exposed increments/precision and reject values that cannot be represented safely. Where the documentation does not state a directional rounding rule, do not invent one that could move a protective level toward additional risk.

9. Costs / commissions / overnight funding

Market details expose instrument overnight-fee metadata, including longRate, shortRate and swapChargeTimestamp in the documented schema. Spread is observable from Bid/Ask (REST uses bid/offer for the current market snapshot).

The public API page reviewed here does not provide a universal REST formula guaranteeing an exact future close commission for every instrument/account before execution. Therefore INITIAL_LONG_CLOSE_COMMISSION and INITIAL_SHORT_CLOSE_COMMISSION must not be fabricated from a generic constant when USE_REAL_COMMISSION is enabled. The implementation should use instrument/account-specific official fee information or a separately validated broker formula.

Likewise, guaranteed-stop premium, conversion charges and realized cost treatment can vary by product/account. These should remain explicit unresolved inputs unless returned by the relevant broker data/history or confirmed by product-specific official terms.

10. Slippage accounting

Deal confirmation exposes an execution level. Current market data exposes Bid and Ask/offer. This is sufficient to record an observed execution-price difference against a locally captured expected quote, but the public API documentation does not prescribe the bot's exact slippage formula for every order type.

For market orders, preserve both expected quote/time and confirmed execution level. If one broker request affects multiple deals, do not invent a single fill price without using the confirmation's actual affected execution information available to the adapter.

11. Account financial state

GET /accounts returns account balances with documented fields such as balance, deposit, profitLoss and available, together with account currency and status. Session responses also include accountInfo. Account activity/history should be used for broker-confirmed operational events instead of relying only on locally calculated P&L.;

12. Error handling principle

The OpenAPI endpoint definitions expose HTTP response classes such as 400 Bad Request, 401 Unauthorized, 404 Not Found and 429 Too Many Requests depending on operation. Trading rejections also surface through deal confirmation/activity status and reason data. CapitalDemoBroker should preserve the raw Capital.com reason/status in logs while mapping it into a smaller internal error taxonomy.

Recommended internal classes: AUTH_REQUIRED, RATE_LIMITED, INVALID_REQUEST, MARKET_NOT_TRADEABLE, BROKER_REJECTED, NOT_FOUND, AMBIGUOUS_OUTCOME, NETWORK_ERROR and RECOVERY_REQUIRED. These are project-side classifications, not claimed Capital.com enum names.

13. Recovery sequence after internet loss

1. Stop new strategy transitions.
2. Restore/recreate REST session if necessary.
3. Fetch active account and verify expected account ID and hedgingMode.

4. Fetch GET /positions.
5. Fetch GET /workingorders.
6. Resolve any persisted dealReference through GET /confirms/{dealReference}.
7. Use account activity/history when a prior operation remains ambiguous.
8. Reconcile SQLite state to broker-authoritative positions/orders.
9. Reconnect WebSocket and resubscribe.
10. Only then resume scenario execution.

14. Items still requiring product/account-specific confirmation

- Exact commission formula for the selected GOLD CFD and account jurisdiction/type.
- Exact guaranteed-stop premium for the selected instrument/account if ever used.
- Exact conversion-fee treatment where account and instrument currencies differ.
- Broker-supported semantics for partial position closes if the strategy needs them.
- Definitive API mechanism for removing only SL or only TP when updating an existing position.
- Any holiday-specific trading-hours exceptions not represented by current market status/opening-hours metadata.

15. Source

Official source: Capital.com Public API 1.0.0, General information, Authentication, Orders and positions, FAQ/API limitations, Accounts/preferences, Account activity history, Markets and WebSocket API; accessed 10 Aug 2026: <https://open-api.capital.com/>